

HTTP 완벽 가이드 · Week 3

TCP 커넥션과 지연의 정체

4.1 ~ 4.2.5 · p.85 ~ 96

발표 3-1 · 3-2 · 정우인

PART 3-1

TCP 커넥션의 기본

4.1 · p.85 ~ 91

지금까지의 HTTP 아래에는 TCP가 있었다

HTTP

지금까지 본 것

메서드 · 상태 코드 · 헤더

TCP

오늘 볼 것

커넥션 · 신뢰성 있는 전송

IP

주소를 보고 목적지까지 패킷 배달

TCP 커넥션이 한 번 맺어지면

클라이언트 ↔ 서버가 주고받는 메시지는

✓ 손실되지 않고

✓ 손상되지 않고

✓ 순서가 바뀌지 않고

→ 안전하게 전달된다

HTTP는 전송의 신뢰성을 직접 챙기지 않는다 — TCP에 맡긴다

브라우저가 URL 하나를 처리하는 7단계

http://

www.joes-hardware.com

:80

/power-tools.html

어디로 연결할지 알아내기

1 URL에서 호스트 명 추출

www.joes-hardware.com

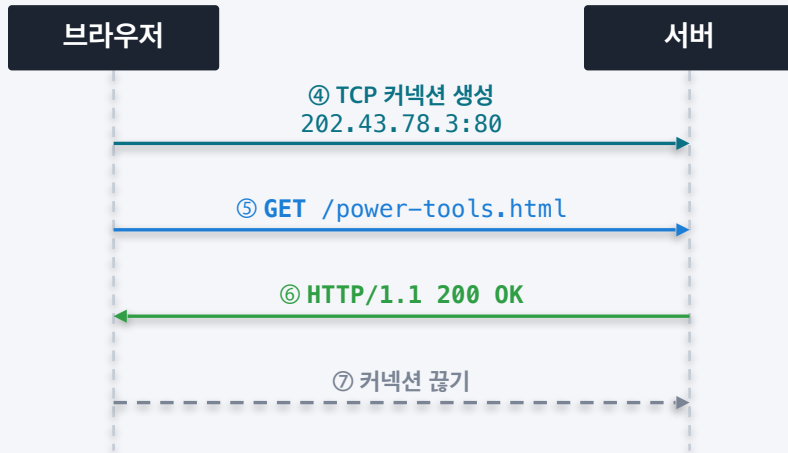
2 호스트 명으로 IP 주소 얻기

202.43.78.3

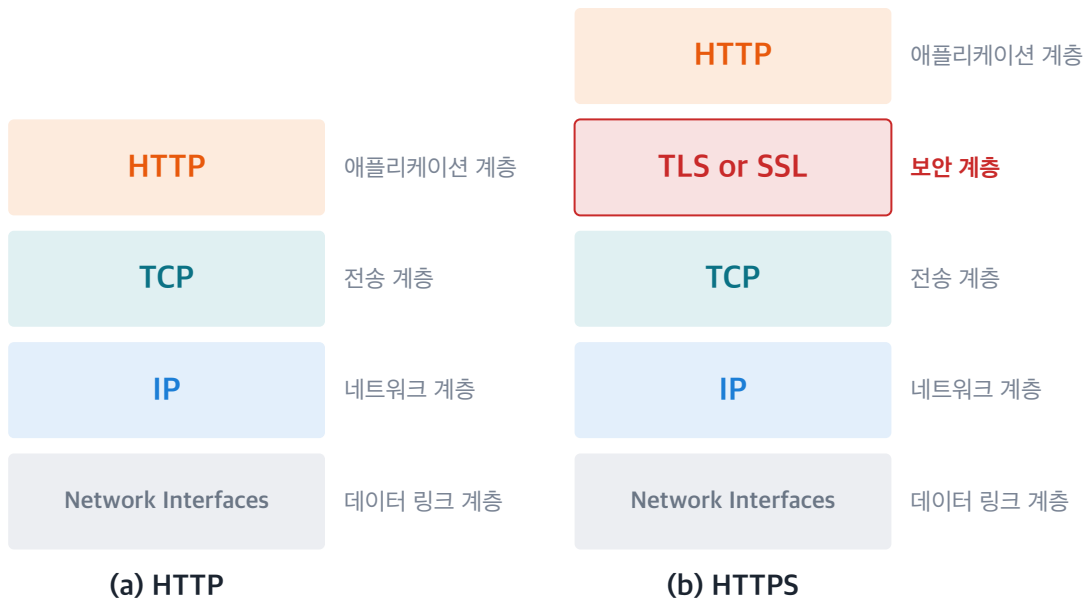
3 포트 번호 얻기

80

연결하고 주고받기



HTTP와 HTTPS의 프로토콜 스택

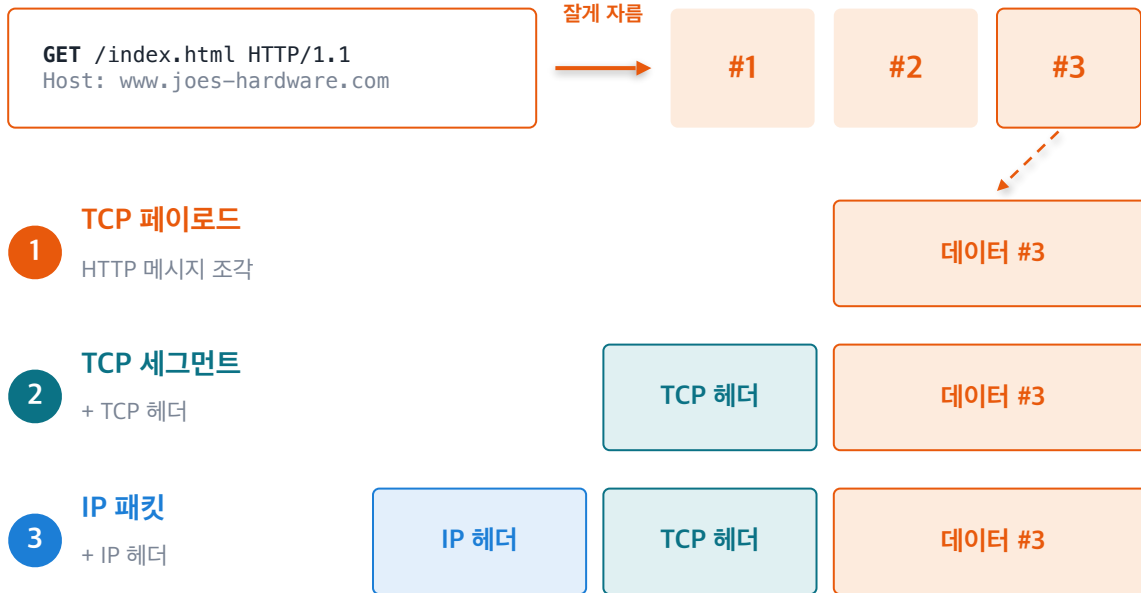


여기서는 이것만

- 1 HTTPS = HTTP와 TCP 사이에 **보안 계층(TLS)**이 하나 더
- 2 HTTP 메시지는 **그대로** TLS가 암호화해서 TCP로 넘긴다
- 3 SSL은 옛 이름, 지금은 TLS
자세한 건 **Week 10 HTTPS**

HTTP 메시지는 잘리고, 헤더가 붙어서 나간다

HTTP 메시지



→ 네트워크 인터페이스를 거쳐 **서버로 전송** · 서버는 반대 순서로 헤더를 벗겨 HTTP 메시지를 다시 조립

헤더에 담기는 것

TCP 헤더

- 발신지 · 목적지 포트
- 순서 번호 · 확인응답
- 플래그 SYN · ACK · FIN ...
- 윈도우 크기 · 체크섬

IP 헤더

- 발신지 · 목적지 IP 주소
- TTL · 상위 계층 프로토콜
- 헤더 체크섬

포트는 TCP, IP 주소는 IP 헤더

소켓 API — TCP/IP의 세부사항을 숨긴다

클라이언트

(C1) IP 주소와 포트를 얻는다

(C2) 새로운 소켓을 생성한다 `socket`

(C3) 서버의 IP:포트로 연결한다 `connect`

(C4) 성공적으로 연결

(C5) HTTP 요청을 보낸다 `write`

(C6) HTTP 응답을 기다린다 `read`

(C7) HTTP 응답을 처리한다

(C8) 커넥션을 닫는다 `close`

서버

(S1) 새로운 소켓을 만든다 `socket`

(S2) 80 포트로 소켓을 묶는다 `bind`

(S3) 소켓 커넥션을 허가한다 `listen`

(S4) 커넥션을 기다린다 `accept`

(S5) 애플리케이션에 커넥션 통지

(S6) 요청을 읽기 시작한다 `read`

(S7) HTTP 요청 메시지를 처리한다

(S8) HTTP 응답을 보낸다 `write`

(S9) 커넥션을 닫는다 `close`

프로그래머는 `socket` · `connect` · `write` · `read` · `close`만 부른다 — 핸드셰이크 · 세그먼트 분할 · 재전송은 OS의 TCP/IP 스택이 처리

포트 · 소켓 · 5-tuple · TCP 커넥션

포트 번호

호스트 안에서
네트워크 통신을 하는
프로세스를 식별

:80

소켓

애플리케이션 계층과
전송 계층 사이의 문
5-tuple로 식별, 2종류

socket()

5-tuple

프로토콜

발신지 IP · 발신지 포트
수신지 IP · 수신지 포트

책은 프로토콜 뺀 4개 값

TCP 커넥션

3-way 핸드셰이크로
두 소켓이 연결되어 생긴
논리적 통로

connect() ↔ accept()

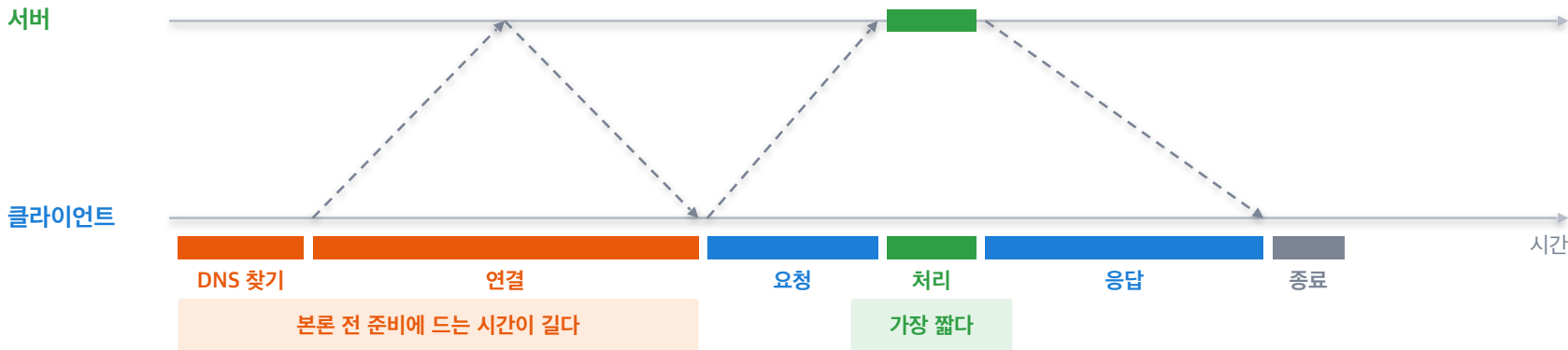
상태	소켓 종류	5-tuple	역할
대기 중	리스닝 소켓	{TCP, *, *, 서버IP, 80}	문을 열고 손님을 기다리는 안내데스크 S1~S4 socket · bind · listen
통신 중	연결된 소켓	{TCP, 클라이언트IP, 클라이언트포트, 서버IP, 80}	특정 손님과 대화하는 전용 상담 창구 accept()가 손님마다 새로 만든다

PART 3-2

HTTP 트랜잭션 지연의 구성

4.2.1 ~ 4.2.5 · p.91 ~ 96

HTTP 트랜잭션은 어디서 시간을 쓰나



DNS 찾기

URL의 호스트 명 → IP 주소 알아내기
캐시에 없으면 DNS 인프라를 거쳐 오래 걸림

TCP 연결

데이터를 보내기 전에
커넥션 설정 패킷이 **한 번 왕복**해야 함

서버 처리

요청을 받아 응답을 만드는 **실제 일**
네트워크에 비하면 상대적으로 짧다

대부분의 HTTP 지연은 서버 처리가 아니라 **TCP 네트워크 지연** 과 **DNS 조회** 에서 생긴다

HTTP를 느리게 만드는 TCP 지연 요소 다섯 가지

1

TCP 커넥션 핸드셰이크

3-way handshake

데이터를 보내기 전에 SYN · SYN+ACK가 먼저 왕복

작은 트랜잭션은 50% 이상을
TCP 구성에 씀

2

확인응답 지연

Delayed ACK

데이터를 받자마자 ACK를 보내지 않고
잠깐 기다렸다가 보냄

ACK가 늦게 도착
→ 보낸 쪽이 기다림

3

TCP 느린 시작

slow start · 혼잡 제어

처음엔 최대 속도를 제한하고
점점 높여 나감 (커넥션이 스스로 튜닝)

새 커넥션은 튜닝된
커넥션보다 느림

4

Nagle 알고리즘 · TCP_NODELAY

자세히는 3-3

작은 데이터를 바로 보내지 않고 모아서 전송
이전 데이터의 ACK가 올 때까지 버퍼에 쌓음

작은 HTTP 메시지가
늦게 나감

5

TIME_WAIT 누적과 포트 고갈

자세히는 3-3

연결을 끊은 쪽이 바로 정리하지 않고 잠깐 대기
늦게 온 패킷이 다음 연결과 섞이지 않게

짧은 연결이 많으면
포트가 바닥남

Keep-Alive와 지속 커넥션 — 연결을 끊지 않고 재사용



HTTP/1.0 Keep-Alive

기본 요청마다 연결을 끊는다

유지하고 싶으면

Connection: keep-alive

“끊지 말자”고 따로 요청해야 함

HTTP/1.1 지속 커넥션 Persistent Connection

기본 연결 유지가 기본값

끊고 싶을 때만

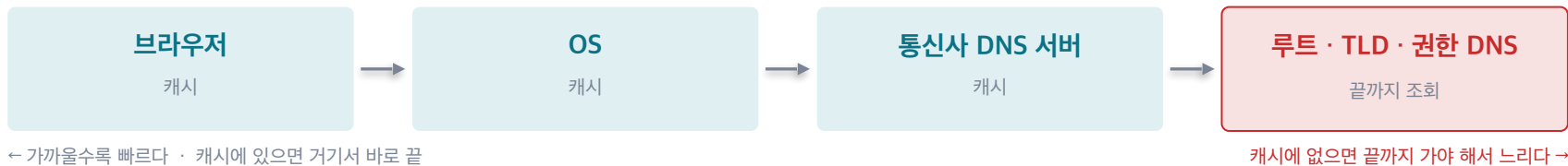
Connection: close

헤더를 안 붙여도 유지된다

둘 다 “연결을 끊지 않고 재사용” — 기본값이 반대일 뿐 · 자세히는 Week 4

DNS 캐싱 — 한 번 알아낸 IP는 잠깐 저장해서 재사용

naver.com 의 IP 주소는? → 가까운 곳부터 차례로 확인



어디에 저장되나

브라우저, OS, 통신사 DNS 서버 등
여러 곳에 나눠서 저장

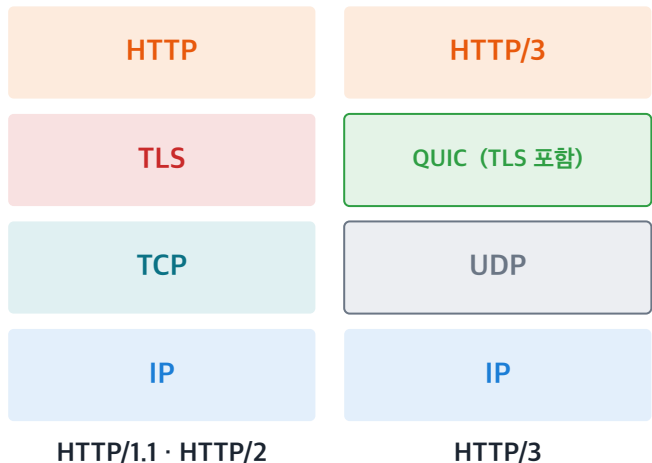
얼마나 저장하나

저장 기간은 TTL로 정해져 있고
지나면 다시 조회한다

naver.com TTL=300 → 5분 동안 재사용

→ DNS 조회 시간 자체를 없애서 자연을 줄인다 (캐시에 있을 때)

HTTP/3 — TCP 대신 UDP 위의 QUIC



TCP + TLS 자리를 QUIC 하나가 맡는다

1 연결이 빠르다

① 핸드셰이크

TCP 핸드셰이크 + TLS 핸드셰이크를 **한 번에**
이전에 접속한 서버면 거의 바로 데이터 전송

2 막힘이 적다

패킷 손실

TCP: 패킷 하나가 손실되면 그 뒤가 **전부 대기**
QUIC: 요청별로 따로 처리 → **손실된 요청만 대기**

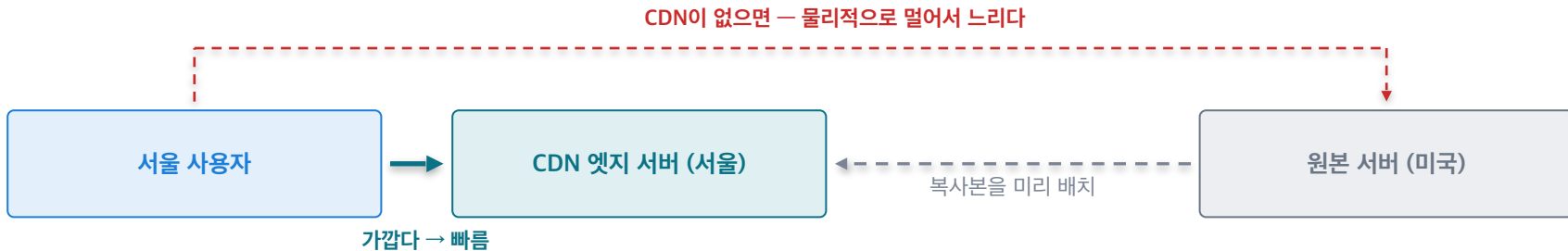
3 네트워크가 바뀌어도 유지

재연결

와이파이 → LTE로 바뀌어도 연결이 끊기지 **않음**
TCP는 IP · 포트 4개 값으로 식별해서 IP가 바뀌면 끊긴다

→ **연결 설정 시간** 과 **패킷 손실로 인한 지연** 을 줄인다

CDN — 사용자와 가장 가까운 서버에서 보내준다



무엇을

이미지 · 동영상 · JS 같은 콘텐츠를
전 세계 여러 서버에 **복사**해 둔다

어떻게 빨라지나

가장 가까운 서버가 응답 → 거리 단축
왕복이 짧아지니 핸드셰이크도 빨라짐

덤

원본 서버의 부담도 줄어든다

예

CloudFront (AWS)

Cloudflare

Akamai

→ 물리적 거리로 인한 **지연**을 줄인다